

FINANCE MANAGEMENT SYSTEM REST API

Complete Project Documentation

Implemented Topics • Modules • Architecture • APIs • Monitoring • Security • Testing • Evidence

This document consolidates the Finance Management System project topics and the implementation/testing work completed in the project.

1. Project Overview

The Finance Management System REST API is a Java and Spring Boot backend application for managing customers, accounts, transactions, loans, loan documents and investments. It uses Spring Data JPA and Hibernate for persistence, MySQL as the relational database, Spring Security with JWT for protected APIs, Swagger/OpenAPI for API documentation, Postman for testing, Spring Boot Actuator for monitoring, HikariCP for database connection pooling, Scheduler for periodic execution, and Spring AOP for cross-cutting concerns.

2. Project Objectives

- Manage customers, accounts, transactions, loans, loan documents and investments.
- Provide RESTful APIs using a layered Controller, Service, Repository and Entity architecture.
- Use MySQL with JPA/Hibernate for persistence and entity relationships.
- Validate incoming request data and handle application exceptions.
- Secure protected APIs using JWT and Spring Security.
- Provide API documentation and testing through Swagger/OpenAPI.
- Implement API versioning using /api/v1.
- Implement pagination for list APIs.
- Use HikariCP connection pooling and Actuator monitoring.
- Implement scheduled processing and provide runtime evidence.
- Implement AOP-based logging, audit logging and execution-time profiling.
- Use one POST method for both create and update instead of separate PUT update methods.
- Handle loan-related file upload, listing, download and deletion.
- Test APIs and monitoring endpoints through Postman and verify runtime behavior in STS.

3. Technology Stack

Technology	Usage
Java	Backend programming language
Spring Boot	Backend application framework
Spring MVC	REST controller and HTTP request handling
Spring Data JPA	Repository and persistence abstraction
Hibernate	Object-relational mapping
MySQL 8	Relational database
Maven	Build and dependency management
Lombok	Reduction of repetitive Java code
Jakarta Validation	Request validation
Spring Security / JWT	Authentication and authorization
Swagger / OpenAPI	API documentation and API testing

Postman	REST API testing
Spring Boot Actuator	Health and monitoring endpoints
HikariCP	Database connection pool
Spring AOP	Logging, audit and profiling
JUnit / Mockito	Unit and service-layer testing

4. Application Architecture

The project follows a layered architecture:

Client / Postman / Swagger → Controller → Service → Repository → JPA/Hibernate → MySQL

Layer	Responsibility
Controller	Receives HTTP requests and returns HTTP responses.
Service	Contains business rules and coordinates operations.
Repository	Provides database access through Spring Data JPA.
Entity	Represents database tables and relationships.
DTO	Defines controlled request and response models.
Security	Handles JWT authentication and authorization.
Aspect	Handles cross-cutting logging, audit and profiling.
Scheduler	Runs scheduled processing.
Config	Contains application and OpenAPI configuration.

5. Main Modules

Module	Purpose
Customer	Customer registration and profile management.
Account	Customer-linked account management and balance information.
Transaction	Deposit, withdrawal and transfer processing.
Loan	Loan creation, approval and repayment.
Loan Document	Upload, list, download and delete loan documents.
Investment	Investment management, maturity and performance information.
Audit Log	Stores selected successful service-operation audit records.

6. Business Flow

Customer Registration → Account Creation → Account Transactions → Loan Management → Investment Management

A customer can have multiple accounts, loans and investments. Accounts can have multiple transactions. Loan documents are associated with loans.

7. Entity Relationships

Parent	Relationship	Child	Meaning
Customer	One-to-Many	Account	One customer can have

			multiple accounts.
Account	One-to-Many	Transaction	One account can have multiple transactions.
Customer	One-to-Many	Loan	One customer can have multiple loans.
Customer	One-to-Many	Investment	One customer can have multiple investments.
Loan	One-to-Many	Loan Document	A loan can have multiple supporting documents.

8. Database Design

Table	Important Fields
customers	customerId, customerName, email, phone, address, dateOfBirth
accounts	accountId, accountNumber, accountType, balance, customerId
transactions	transactionId, transactionType, amount, transactionDate, accountId, targetAccountId
loans	loanId, loanType, loanAmount, interestRate, durationMonths, customerId, startDate, status, totalPayable
loan_documents	documentId, fileName, filePath, contentType, documentType, loanId
investments	investmentId, investmentType, amount, currentValue, investedDate, maturityDate, status, customerId
audit_logs	auditId, action, entityName, entityId, timestamp, details

The project uses the finance_db MySQL schema and SQL/database verification was used during development.

9. REST API Versioning

The implemented finance APIs use the versioned /api/v1 base path.

Module	Implemented Endpoint
Customer	/api/v1/customers
Account	/api/v1/accounts
Transaction	/api/v1/transactions
Loan	/api/v1/loans
Loan Documents	/api/v1/loans/{loanId}/documents
Investment	/api/v1/investments
Audit Logs	/api/v1/audit-logs
Authentication	/api/auth/login
Actuator	/actuator/...

10. Create + Update Using the Same POST Method

The implemented controllers use one POST operation for both creation and update. Separate PUT update methods were removed from the implemented finance controllers.

Request Condition	Operation
ID is null	Create a new record
Existing ID is provided	Update the existing record

Example: POST /api/v1/customers with customerId=null creates a customer. POST /api/v1/customers with an existing customerId updates that customer.

11. Pagination

Pagination is implemented for Customer, Account, Transaction, Loan and Investment list APIs using Spring Data Page and PageRequest.

Example: GET /api/v1/customers?page=0&size=5

page selects the page number and size controls the number of records requested in that page.

12. Actuator

Spring Boot Actuator is configured to expose health, info and metrics.

Endpoint	Purpose
/actuator/health	Application and database health
/actuator/info	Application information
/actuator/metrics	Available application and system metrics

Evidence: the health endpoint returned overall status UP and the MySQL database component status UP.

13. Actuator Metrics

GET /actuator/metrics was tested in Postman and returned HTTP 200 OK. The response contained application, HTTP, JVM, database and HikariCP metric names.

- hikaricp.connections.active
- hikaricp.connections.idle
- hikaricp.connections.max
- hikaricp.connections.min
- hikaricp.connections.acquire
- hikaricp.connections.creation
- hikaricp.connections.pending
- hikaricp.connections.timeout
- hikaricp.connections.usage

This provides evidence that Actuator Metrics is active and exposing runtime measurements.

14. HikariCP / Connection Pool

HikariCP is used as the database connection pool. The configured pool name is FinanceHikariPool.

Metric	Verified Value
Pool name	FinanceHikariPool
Active connections	0.0
Idle connections	2.0
Maximum connections	10.0
Minimum connections	2.0

The application startup console also showed the FinanceHikariPool starting, adding a database connection and completing successfully.

15. Scheduler

Scheduling is enabled in the Spring Boot application. FinanceScheduler executes every 60 seconds.

Verified console evidence included repeated messages: Finance scheduler executed successfully.

The STS console showed repeated FinanceScheduler executions at approximately one-minute intervals.

16. AOP

Spring AOP is implemented around service methods. The service logging aspect intercepts service execution and records execution time. A separate audit aspect records selected successful service operations.

- Service method interception
- Around advice
- Execution-time measurement
- Cross-cutting logging
- Audit interception

17. Logger

SLF4J logging through Lombok @Slf4j is used for runtime messages.

- Scheduler execution messages appear in the STS console.
- Service execution messages appear in the STS console.
- Example verified message: Service method: save executed in 12 ms.

18. Profiling / Execution Time

Profiling is implemented through the AOP service logging aspect. Service execution time is calculated in milliseconds and written to the application log.

Verified Console Output	Meaning
Service method: save executed in 12 ms	Service execution time measured
Service method: save executed in 9 ms	Another service execution time measured

19. Audit Logging

The application contains AuditLog entity, repository, service and controller components. AuditAspect records selected successful save, delete, approve and repay service operations.

GET /api/v1/audit-logs was tested in Postman and returned audit records.

Field	Verified Example
action	SAVE
entityName	CustomerService
entityId	null in the verified records
timestamp	Recorded at operation time
details	Service operation executed successfully

20. Swagger / OpenAPI

Swagger/OpenAPI provides browser-based API documentation and testing. The verified Swagger page displayed the Finance Management System API, version v1, OAS 3.1, versioned endpoints and a Bearer JWT Authorize control.

Swagger UI endpoint: <http://localhost:8080/swagger-ui/index.html>

21. JWT / Spring Security

Spring Security protects the finance APIs using JWT authentication.

Step	Action
1	POST /api/auth/login with valid credentials.
2	Receive the JWT token.
3	Set Authorization type to Bearer Token in Postman.
4	Call a protected /api/v1 endpoint.
5	Successful response verifies authenticated API access.

A protected POST /api/v1/customers request was successfully tested with a Bearer token.

22. File Handling

Loan document file handling is implemented using multipart form-data.

Operation	Endpoint
Upload	POST /api/v1/loans/{loanId}/documents
List	GET /api/v1/loans/{loanId}/documents
Download	GET /api/v1/loans/{loanId}/documents/{documentId}/download
Delete	DELETE /api/v1/loans/{loanId}/documents/{documentId}

This supports loan-related document management such as identity, address, income and loan documents according to the project requirement.

23. Postman Testing

Postman is used for API testing, authorization, pagination, Actuator metrics, audit logs and finance operations.

1. Start the Spring Boot application.
2. Login and obtain the JWT token.
3. Use Bearer Token authorization for protected APIs.
4. Test create with POST and a null ID.
5. Test update with the same POST and an existing ID.
6. Test GET and DELETE operations.
7. Test pagination with page and size.
8. Test loan approval and repayment.
9. Test loan document upload, list, download and delete.
10. Test audit logs.
11. Test Actuator health and metrics.

24. DTO Pattern

Request and response DTOs separate the REST API contract from JPA entities. Request DTOs contain validation rules while response DTOs expose controlled data.

25. Validation

- @NotBlank validates required text values.
- @Email validates email format.
- @NotNull validates required values.
- @Positive validates positive numeric values.
- @Valid triggers DTO validation.

26. Exception Handling

The application uses ResourceNotFoundException and centralized exception handling for missing resources, validation failures and unexpected errors.

Error Type	Typical HTTP Status
Invalid input / validation failure	400 Bad Request
Requested resource not found	404 Not Found
Unexpected application/server error	500 Internal Server Error

27. JPA / Hibernate

Spring Data JPA repositories provide database access and Hibernate performs object-relational mapping. Entity relationships connect customers with accounts, loans and investments; accounts with transactions; and loans with documents.

28. MySQL and SQL

MySQL 8 is used with the finance_db schema. SQL schema and database operations support customers, accounts, transactions, loans, loan_documents, investments and audit_logs.

29. Transaction Management

Financial operations use transaction management where multiple related database changes must remain consistent. Transaction processing validates the transaction type and amount before applying account balance changes.

30. Spring Core and Dependency Injection

Spring dependency injection connects controllers, services, repositories, aspects, schedulers and supporting components through Spring-managed beans.

31. Lombok

Lombok is used for getters, setters, constructors, builders, required constructors and logging setup, reducing repetitive Java code.

32. Unit Testing

JUnit and Mockito are used for service-layer testing and important application behavior. Tests cover business operations and failure scenarios where applicable.

33. Complete API Operations

Module	Operations
Customer	POST save/create-update, GET all, GET by ID, DELETE
Account	POST save/create-update, GET all, GET by ID, DELETE
Transaction	POST save/create-update, GET all, GET by ID, DELETE
Loan	POST save/create-update, GET all, GET by ID, PATCH approve, POST repay, DELETE
Loan Document	POST upload, GET list, GET download, DELETE
Investment	POST save/create-update, GET all, GET by ID, DELETE
Audit Log	GET audit records

34. Evidence and Verification Summary

Topic	Evidence / Verification	Status
Actuator	Health endpoint returned UP	Completed
Actuator Metrics	/actuator/metrics returned metric names with HTTP 200	Completed
HikariCP / Connection Pool	FinanceHikariPool and active/idle/max/min metrics verified	Completed

Scheduler	Repeated FinanceScheduler console executions	Completed
AOP	Service interception and execution-time logging verified	Completed
Logger	STS runtime logs verified	Completed
Audit Logging	Audit records returned by /api/v1/audit-logs	Completed
Swagger / OpenAPI	Swagger UI and Bearer Authorize control verified	Completed
API Versioning	Finance endpoints use /api/v1	Completed
Pagination	Page and size implemented for list APIs	Completed
Profiling / Execution Time	save execution times shown in milliseconds	Completed
Same POST Create/Update	Null ID and existing ID flow implemented/tested	Completed
JWT / Spring Security	Bearer-token protected API tested	Completed
File Handling	Upload/list/download/delete flow implemented	Completed
Postman Testing	Finance and monitoring endpoints tested	Completed
Database Design	Finance tables and relationships implemented	Completed
Modules / Architecture / DTO / Validation / Exceptions / JPA / Hibernate / MySQL	Implemented in project	Completed
Unit Testing	JUnit / Mockito tests included	Completed

35. Final Topic Checklist

12. Actuator & Actuator Metrics
13. HikariCP / Connection Pool
14. Scheduler
15. AOP
16. Logger
17. Audit Logging
18. Swagger / OpenAPI
19. API Versioning
20. Pagination
21. Profiling / Execution Time
22. Create + Update using the Same POST Method
23. JWT / Spring Security
24. File Handling
25. Postman Testing
26. Database Design
27. Modules, Architecture, DTO, Validation, Exceptions, JPA/Hibernate, MySQL and related concepts
28. Testing and Evidence Summary

36. Runtime Evidence Summary

- Application started successfully on port 8080.

- Tomcat started successfully on port 8080.
- MySQL connection was established through FinanceHikariPool.
- Actuator health reported UP.
- Actuator metrics returned HikariCP and application metrics.
- FinanceScheduler executed repeatedly.
- AOP service execution logs appeared in the STS console.
- Profiling output displayed service execution time in milliseconds.
- Audit records were returned through the audit API.
- Swagger displayed versioned APIs and Bearer authorization.
- Protected API access succeeded using a JWT Bearer token.
- Postman successfully tested the relevant API and monitoring flows.

37. Conclusion

The Finance Management System REST API combines the required finance-domain modules with the Spring Boot concepts implemented during the project. The final implementation covers Actuator and Actuator Metrics, HikariCP connection pooling, Scheduler, AOP, Logger, Audit Logging, Swagger/OpenAPI, API Versioning, Pagination, Profiling, single-POST create/update, JWT/Spring Security, File Handling, Postman testing, database design, layered architecture, DTOs, validation, exception handling, JPA/Hibernate, MySQL, transaction management and testing.

38. Learning Outcomes

The project provided practical experience in building and verifying a complete backend REST API using Java and Spring Boot.

- Java and object-oriented programming in backend development.
- Spring Core dependency injection and Spring-managed beans.
- Spring Boot configuration and application startup.
- Spring MVC and REST API development.
- DTO-based request and response design.
- Jakarta Bean Validation.
- Service-layer business logic and layered architecture.
- Spring Data JPA and Hibernate persistence.
- MySQL database design, SQL and database verification.
- Financial transaction processing.
- Loan approval and repayment processing.
- Loan document file handling.
- JWT authentication and Spring Security.
- Swagger/OpenAPI documentation.
- Actuator monitoring and metrics.
- HikariCP connection pooling.
- Scheduler implementation and runtime evidence.
- AOP, logging, audit logging and profiling.
- Pagination and API versioning.
- Postman API testing.
- JUnit and Mockito testing.
- Debugging application, database and schema issues.